

Game Programming Patterns

L'Architettura del Software è (Gestire) il Cambiamento

La vera misura di una buona architettura non è quanto sia complessa o ricca di pattern accademici, ma **quanto facilmente accoglie i cambiamenti**. Se un programma non deve mai essere modificato, la sua architettura interna è di fatto irrilevante.

- **Il collo di bottiglia umano:** Prima di poter modificare il codice (per aggiungere una feature o correggere un bug), devi "caricare" l'intero contesto nel tuo cervello. Questo processo di lettura e comprensione è spesso la fase più lenta e faticosa della programmazione.
- **L'obiettivo:** Una buona architettura riduce al minimo la quantità di informazioni che devi tenere a mente contemporaneamente per poter lavorare su una specifica porzione di codice.

Il Potere (e il Pericolo) del Disaccoppiamento

Il disaccoppiamento (decoupling) serve esattamente a isolare le parti del sistema. Se due moduli sono disaccoppiati, puoi comprenderne e modificarne uno senza dover studiare l'altro.

- **L'entusiasmo:** Lavorare su codice ben disaccoppiato è un'esperienza eccellente; i cambiamenti si inseriscono perfettamente senza creare "effetti a catena" indesiderati e bug imprevisti.
- **Il costo nascosto (YAGNI):** Ogni livello di astrazione aggiunge complessità. Cercare di prevedere il futuro inserendo interfacce e punti di estensione "perché potrebbero servire" (speculazione) spesso crea un labirinto strutturale incomprensibile. Il mantra per combattere questa tendenza è **YAGNI** (*You Aren't Gonna Need It* - Non ne avrai bisogno): non aggiungere architettura finché non è strettamente necessaria.

Flessibilità vs. Performance

Nel game development c'è una tensione costante tra l'astrazione e l'ottimizzazione del codice a basso livello.

- **L'architettura rimuove le assunzioni:** L'uso di interfacce, sistemi di messaggistica e metodi virtuali rende il codice flessibile proprio perché non sa a priori con quale entità esatta interagirà a runtime.
- **La performance si nutre di assunzioni:** L'ottimizzazione pura si ottiene quando conosciamo i limiti esatti (es. "avremo sempre massimo 256 nemici dello stesso tipo", il che ci permette di impacchettarli in un singolo blocco di memoria contigua per la CPU).
- **Il compromesso:** È più facile rendere veloce un gioco divertente, piuttosto che rendere divertente un gioco veloce. Mantieni il codice flessibile durante le iterazioni per trovare il giusto game design, e ottimizza "strappando via" le astrazioni solo successivamente, quando il design è consolidato.

Prototipazione e il Codice "Spazzatura"

Il game design richiede tantissima sperimentazione. Scrivere codice "alla carlona" (slapdash) per testare velocemente un'idea di gameplay è una pratica perfettamente legittima, ma ha una regola d'oro: **deve poter essere buttato via**.

- Il rischio più letale è che un prototipo instabile e non testato venga poi "ripulito alla buona" per diventare il codice definitivo, condannando il progetto a un debito tecnico costante.

La Semplicità come Vera Soluzione

Il modo migliore per bilanciare velocità di sviluppo a lungo termine, performance in esecuzione e velocità di rilascio a breve termine è perseguire la **semplicità**.

Non significa impiegare *meno tempo* per programmare, ma dedicare tempo a distillare il problema fino a trovare la soluzione più pulita e diretta possibile. Meno codice "impalcatura" significa meno overhead cognitivo per capirlo e, molto spesso, maggiore velocità della CPU perché c'è fisicamente meno logica da eseguire.

Summary

- **Architettura = Sopravvivenza:** Serve a far sì che il tuo cervello non esploda quando devi modificare il codice tra sei mesi.
- **Disaccoppiamento:** Dividi il gioco in scompartimenti stagni. Se modifichi il movimento, non devi rischiare di rompere il menu principale.
- **KISS & YAGNI:** Mantieni le cose stupide e semplici. Non costruire sistemi iper-complessi "perché in futuro chissà, magari mi serviranno". Se non ti serve *oggi*, non scriverlo.
- **Performance vs Flessibilità:** Il codice flessibile (astratto) fa prototipare in fretta ma gira più lentamente. Il codice ottimizzato gira veloce ma è rigido. Prototipa flessibile, ottimizza solo alla fine.

Design Patterns Revisited

I 6 Pattern Selezionati

L'autore del libro sceglie di analizzare 6 pattern classici rivisitandoli con l'esperienza di oggi e, soprattutto, applicandoli al **game development**:

- **Singleton**: Considerato dall'autore troppo abusato (spesso diventa una scusa per scrivere codice disordinato).
- **Command**: Un pattern molto potente ma ingiustamente sottovalutato.
- **Flyweight e Observer**: Due pattern fondamentali e onnipresenti nello sviluppo di videogiochi per gestire ottimizzazione e disaccoppiamento.
- **Prototype e State**: Pattern interessanti per capire come le architetture si evolvono nel panorama generale della programmazione.

Command

Il Pattern Command in Breve

L'autore considera il **Command** uno dei pattern più potenti per districare architetture complesse. Critica la definizione accademica originale (troppo confusa e astratta) e ne propone una molto più pragmatica:

Un comando è una chiamata a metodo "reificata" (resa reale).

- **Cosa significa "Reificare"**? Significa prendere un concetto astratto (come l'esecuzione di una funzione) e trasformarlo in un pezzo di **dati**, ovvero un oggetto tangibile all'interno del codice.
- **A cosa serve?** Invece di eseguire un'azione immediatamente (es. `Salta()`), crei un oggetto che *rappresenta* quell'azione (es. `ComandoSalto`). Questo oggetto può essere salvato in una variabile, passato come parametro, messo in coda o annullato.
- In pratica, il Command è la versione orientata agli oggetti di un *callback* o di un puntatore a funzione.

L'analogia del Ristorante: Pensa a quando vai al ristorante. Quando dici al cameriere "Voglio una pizza", lui non corre in cucina a prepararla (esecuzione diretta). Invece, scrive la tua richiesta su un foglietto. Quel foglietto è l'azione "reificata": è un oggetto fisico (il Command) che può essere messo in una coda di comandi, passato a diversi cuochi o perfino cancellato se cambi idea.

Il Livello di Indirizione (Command Pattern)

Per permettere la riconfigurazione dei tasti, dobbiamo trasformare la chiamata diretta `Jump()` in un **oggetto** che possiamo salvare in una variabile e scambiare a piacimento.

Perché scriverlo così: Un'interfaccia stabilisce un contratto. Qualsiasi classe la implementi *deve* avere un metodo `Execute()`. Questo permette al nostro `InputHandler` di trattare tutti i comandi allo stesso modo, senza sapere cosa facciano internamente (Polimorfismo).

1. Il Contratto (L'Interfaccia)

In C#, invece di una classe base con metodi virtuali come in C++, usiamo quasi sempre un'interface.

```
// Interfaccia che definisce cosa deve fare un "Comando"

public interface ICommand

{

    void Execute();

}
```

2. I Comandi Concreti (Le Azioni Reificate)

Ora creiamo delle classi specifiche per ogni azione.

```
public class JumpCommand : ICommand

{

    public void Execute()

    {

        // Qui chiamiamo l'effettiva logica di gioco

        Debug.Log("Il giocatore salta!");

        // Es. Player.Instance.Jump();

    }

}

public class FireCommand : ICommand

{

    public void Execute()

    {

        Debug.Log("Il giocatore spara!");

    }

}
```

3. Il Nuovo Input Handler (Disaccoppiato)

```
public class InputHandler : MonoBehaviour

{

    // Variabili che contengono i NOSTRI COMANDI, non azioni fisse

    private ICommand buttonX;

    private ICommand buttonY;


    private void Start()

    {

        // Fase di "Binding": associamo i comandi ai pulsanti.

        // Questo potrebbe essere letto da un file di salvataggio delle opzioni!

        buttonX = new JumpCommand();

        buttonY = new FireCommand();

    }


    private void Update()

    {

        // L'input chiama il comando, qualunque esso sia in quel momento

        if (Input.GetKeyDown(KeyCode.X)) buttonX.Execute();

        else if (Input.GetKeyDown(KeyCode.Y)) buttonY.Execute();

    }

}
```

Il Problema: Comandi troppo Accoppiati all'Esecutore

Nel passo precedente, il nostro `JumpCommand` faceva saltare *sempre e solo il giocatore*, perché sapeva internamente chi chiamare (es. un ipotetico Singleton `Player.Instance.Jump()`). Questo lo rende rigido. Cosa facciamo se vogliamo che un nemico salti, o se il giocatore prende il controllo di un drone? Dovremmo scrivere un `DroneJumpCommand`? Assurdo.

La Soluzione: Passare l'Attore (Actor) al Comando

La soluzione, come hai intuito nella sfida precedente, è modificare l'interfaccia affinché il comando riceva il suo "bersaglio" (l'Actor) al momento dell'esecuzione. Invece di dirgli *chi* far saltare, il comando sa solo *come* far saltare qualcuno, e noi gli passiamo il soggetto al momento opportuno.

Il Cambio di Paradigma: Da Azioni Generiche ad Azioni Specifiche(Undo/Redo)

Nei passaggi precedenti, usavamo il Command come un bottone riutilizzabile: un singolo `JumpCommand` in memoria veniva chiamato ogni volta che si premeva 'X'.

Per implementare l'Undo, c'è un cambio fondamentale di architettura: **il Comando non è più generico, ma diventa un'istanza unica legata a uno specifico momento nel tempo e a uno specifico bersaglio**. Usando la nostra analogia del ristorante: prima il comando era la voce "Pizza" sul menu. Ora il comando è "Il Tavolo 4 ha ordinato una Pizza alle 20:30". Se il Tavolo 4 cambia idea, possiamo annullare *quell'ordine specifico*.

Per fare questo, il comando deve possedere una **memoria di stato**: deve ricordarsi com'era il mondo un attimo prima di essere eseguito, per poterlo ripristinare.

Analisi Architetture: Dal Paradigma Oggetto al Paradigma Funzionale

Chiusure (Closures) come Comandi Invisibili

L'autore C++ esprime quasi "invidia" verso i linguaggi che supportano funzioni di prim'ordine (first-class functions) e chiusure (closures). La buona notizia? C# le supporta benissimo tramite **Delegati e Action/Func**.

Nel pattern Command puro (OOP), creavamo un'intera classe `MoveUnitCommand` che incapsulava lo stato (chi, dove) per poi eseguirlo o annullarlo.

Con un approccio **funzionale (Closure)**, possiamo usare i tipi generici di C# per evitare di scrivere centinaia di classi diverse! Il compilatore C# "sotto il cofano" crea automaticamente le classi necessarie per conservare lo stato della variabile al momento della creazione (ecco cos'è una closure).

```
using System;
using UnityEngine;

// Invece di una ICommand complessa, usiamo due semplici Action (Delegati).
// Questo è il "Comando Funzionale"
public class ActionCommand
{
    public Action ExecuteLogic { get; set; }
    public Action UndoLogic { get; set; }

    public void Execute() => ExecuteLogic?.Invoke();
    public void Undo() => UndoLogic?.Invoke();
}

public class InputHandler : MonoBehaviour
{
    // Creiamo il comando al volo usando Lambda Expressions e Closures
    public ActionCommand CreateMoveCommand(GameActor unit, Vector3 destination)
    {
        // Questo è il cuore della Closure: stiamo 'congelando'
        // unit, destination e previousPos all'interno di questi delegati.
        Vector3 previousPos = unit.transform.position;

        return new ActionCommand
        {
            ExecuteLogic = () => { unit.transform.position = destination; },
            UndoLogic = () => { unit.transform.position = previousPos; }
        };
    }
}
```

Flyweight

Immagina di dover descrivere a un amico una foresta. Non gli descriverai la forma di ogni singola foglia di ogni singolo albero; gli dirai "ci sono mille querce", e poi magari gli indicherai dove si trovano le più grandi. Il Flyweight fa esattamente questo, ma per la **memoria del tuo computer**.

Il Problema: Il Collo di Bottiglia del Bus

Renderizzare una foresta lussureggiante in tempo reale è un incubo per le performance. Il problema non è solo la quantità di memoria necessaria per immagazzinare migliaia di alberi (ognuno con le sue mesh e texture ad alta risoluzione), ma il **trasferimento di questi dati**.

Spostare questa mole di informazioni dalla CPU alla GPU 60 volte al secondo intasa il **"bus"** (il canale di comunicazione tra i due processori).

La Soluzione: Il Pattern Flyweight

L'intuizione alla base del pattern è semplice: in una foresta di migliaia di alberi, la maggior parte di essi condivide la stessa identica geometria (Mesh) e le stesse immagini (Texture). Il pattern suggerisce di dividere l'oggetto "Albero" in due parti:

- **Stato Intrinseco (Condiviso):** I dati pesanti uguali per tutti (Mesh, Texture). Ne teniamo una sola copia in memoria.
- **Stato Estrinseco (Specifico dell'istanza):** I dati leggeri unici per ogni albero (Posizione, Scala, Colore).

L'Approccio Flyweight in Unity (Scriptable Objects & Structs)

Per separare i dati condivisi da quelli specifici, in Unity usiamo uno strumento potentissimo: gli **Scriptable Objects**.

I Flyweight devono essere immutabili. Se modifichi il costo di movimento dell'istanza "Erba", stai modificando contemporaneamente tutte le caselle d'erba del mondo!

Scriptable Object

Uno ScriptableObject è un Flyweight naturale — dati condivisi tra N istanze senza duplicazione.

ItemSO (ScriptableObject) ← stato intrinseco (condiviso)

```
|— itemName = "Spada"  
|— damage = 10  
|— icon = sprite
```

Enemy_01 → riferimento a ItemSO ← stato estrinseco (unico per istanza)

Enemy_02 → riferimento a ItemSO

Enemy_03 → riferimento a ItemSO

Tutti e tre i nemici **condividono** lo stesso **ItemSO** in memoria. Se cambi **damage**, cambia per tutti.

La distinzione chiave del pattern:

	Flyweight chiama...	Esempio
Stato intrinseco	uguale per tutte le istanze	mesh, texture, statistiche base
Stato estrinseco	unico per ogni istanza	posizione, scala, colore

Class vs Struct

- `class` → riferimento → **un'unica copia in memoria**, tutti puntano alla stessa
- `struct` → valore → **copiata per ogni istanza**

Quindi sembrerebbe che la `class` sia meglio per risparmiare memoria — e infatti è esattamente quello che fa lo `ScriptableObject` per lo stato intrinseco.

Allora perché `struct` per lo stato estrinseco?

Perché lo stato estrinseco è **per definizione diverso per ogni istanza** — posizione, scala, colore di ogni singolo albero. Non ha senso dividerlo.

E una `struct` di dati piccoli (3 float per la posizione) è:

- allocata nello stack, non nell'heap
 - nessun overhead del Garbage Collector
 - cache-friendly per la GPU
-

Quindi la divisione corretta è:

	Tipo	Perché
Stato intrinseco	<code>ScriptableObject</code> (class)	Condiviso per riferimento — una copia sola
Stato estrinseco	<code>struct</code>	Leggero, unico per istanza, GC-free

Observer

Hai un sistema di achievement ("Cadi da un ponte", "Uccidi 100 nemici"). Questi achievement sono triggerati da sistemi diversi — fisica, combattimento, navigazione. Se scrivi la logica degli achievement direttamente dentro ogni sistema, il codice diventa accoppiato e impossibile da mantenere.

La Soluzione

Un sistema **annuncia** che qualcosa è successo. Chi è interessato si **registra** per ricevere la notifica. Chi annuncia non sa chi riceve.

I Due Attori

Observer — chi vuole essere notificato. Implementa un'interfaccia.

Subject — chi genera le notifiche. Mantiene una lista di observer e li notifica tutti.

La lista è fondamentale: se il subject supportasse un solo observer, il secondo registrato disabiliterebbe il primo.

Problemi Noti

Destroying subjects and observers — se distruggi un observer senza deregistrarlo, il subject ha ancora un puntatore a memoria deallocata. Crash garantito. La regola è: **deregistra sempre prima di distruggere.**

Lapsed Listener Problem — in linguaggi con garbage collector, se non deregistri un observer, il GC non lo raccoglie mai perché il subject tiene ancora un riferimento. Accumuli oggetti zombie in memoria.

Observer sincrono — il subject chiama gli observer direttamente e aspetta che finiscano prima di riprendere. Un observer lento blocca il subject.

Debug difficile — con accoppiamento esplicito vedi subito chi chiama chi. Con observer, devi sapere chi è registrato a runtime per capire il flusso. Più difficile da tracciare.

Quando Usarlo

Quando due sistemi sono **quasi completamente non correlati** e vuoi il minimo di comunicazione tra loro. Achievement e fisica sono l'esempio perfetto. Non usarlo dentro un singolo sistema coeso — lì è meglio un accoppiamento esplicito.

Observer Moderno in C#

Il testo nota che oggi, invece di implementare un'intera interfaccia, si usano direttamente delegati ed eventi — che C# ha integrato nel linguaggio.

Architettura

GameManager

- └─ registra gli Observer sul Subject (Start)
- └─ deregistra gli Observer (OnDestroy)

Subject (PhysicsSystem)

- └─ mantiene la lista degli Observer
- └─ chiama Notify() quando succede qualcosa

Observer (Achievements, AudioSystem, UI...)

- └─ riceve OnNotify() e reagisce

Il Subject non sa chi lo ascolta. Gli Observer non sanno chi li notifica. Il GameManager è l'unico che conosce entrambi e li collega.

Prototype

Il Limite del Pattern

Il pattern risolve il problema degli spawner duplicati, ma sposta il problema: invece di implementare una classe spawner per ogni nemico, devi implementare `Clone()` in ogni nemico. Il codice totale è quasi lo stesso.

In più, `Clone()` nasconde problemi semantici: se un Demon tiene in mano un'ascia, clonare il Demon clona anche l'ascia? Deep copy o shallow copy?

Il Prototype pattern come descritto dal GoF è raramente la soluzione migliore in pratica. Le spawn functions o i generics risolvono lo stesso problema con meno complessità. In Unity specificamente, il pattern non è quasi mai usato in questa forma — si usano Prefab e `Instantiate()` che sono concettualmente la stessa idea ma integrata nel engine.

Self — OOP Senza Classi

Negli anni '80 viene creato **Self**, un linguaggio OOP che non ha classi. L'idea centrale: in un linguaggio classico, stato e comportamento sono separati — lo stato vive nell'istanza, i metodi vivono nella classe. Self elimina questa distinzione: ogni oggetto contiene direttamente sia dati che metodi.

Per riutilizzare comportamento senza classi, Self usa la **delega** — se un oggetto non ha un campo o metodo, lo cerca nel suo oggetto padre. È l'equivalente dell'ereditarietà, ma tra oggetti invece che tra classi.

Per creare nuovi oggetti, Self usa il **clone** — esattamente il Prototype Pattern, ma integrato nel linguaggio stesso. Non devi implementare `Clone()` — ogni oggetto lo supporta automaticamente.

Prototipizzazione dei Dati

Nei giochi moderni, la maggior parte del contenuto vive nei **dati**, non nel codice. Ma i dati hanno gli stessi problemi del codice: duplicazione, manutenzione difficile, nessuna astrazione.

```
{ "name": "goblin grunt", "minHealth": 20, "maxHealth": 30,
  "resists": ["cold", "poison"], "weaknesses": ["fire", "light"] }

{ "name": "goblin wizard", "minHealth": 20, "maxHealth": 30,
  "resists": ["cold", "poison"], "weaknesses": ["fire", "light"],
  "spells": ["fire ball", "lightning bolt"] }

{ "name": "goblin archer", "minHealth": 20, "maxHealth": 30,
  "resists": ["cold", "poison"], "weaknesses": ["fire", "light"],
  "attacks": ["short bow"] }
```

Duplicazione pura. Se vuoi rendere tutti i goblin più forti, devi aggiornare tre posti.

Delega nei Dati

Aggiungi un campo **prototype** che indica a quale oggetto delegare i campi mancanti:

```
{ "name": "goblin grunt", "minHealth": 20, "maxHealth": 30,  
  "resists": ["cold", "poison"], "weaknesses": ["fire", "light"] }  
  
{ "name": "goblin wizard", "prototype": "goblin grunt",  
  "spells": ["fire ball", "lightning bolt"] }  
  
{ "name": "goblin archer", "prototype": "goblin grunt",  
  "attacks": ["short bow"] }
```

Wizard e Archer ereditano health, resists e weaknesses dal grunt. Definiscono solo ciò che li differenzia.

L'Equivalente con ScriptableObject

[CreateAssetMenu]

```
public class MonsterDataSO : ScriptableObject  
{  
    public string monsterName;  
    public int minHealth;  
    public int maxHealth;  
    public string[] resists;  
    public string[] weaknesses;  
}
```

```
// GoblinArcher usa GoblinGrunt come base  
// e aggiunge solo ciò che lo differenzia  
[CreateAssetMenu]
```

```
public class ArcherDataSO : MonsterDataSO  
{  
    public string[] attacks;  
}
```

Le alternative

Spawn functions, generics, o semplicemente `Instantiate()` in Unity — risolvono lo stesso problema con meno complessità. Il valore del capitolo è capire il *concetto* di clonazione come meccanismo, non applicare il pattern alla lettera.

Singleton

Il Singleton è così abusato nell'industria dei videogiochi che la maggior parte del capitolo è dedicata ad evitarlo.

Cos'è il Singleton

Design Patterns lo definisce in due parti:

"Assicura che una classe abbia una sola istanza, e fornisce un punto di accesso globale ad essa."

Una Sola Istanza

Ha senso quando una classe interagisce con un sistema esterno che ha il proprio stato globale. Esempio: un wrapper al filesystem. Se hai operazioni asincrone concorrenti — crea file, cancella file — tutte devono coordinarsi attraverso la stessa istanza. Due istanze separate non si vedono l'una con l'altra.

Accesso Globale

Se i vari sistemi del gioco — logging, salvataggio, caricamento contenuti — non possono creare la propria istanza, come la ottengono? Il Singleton risolve anche questo: espone un metodo statico accessibile da ovunque nel codice.

Perché Lo Usiamo

1. Lazy Initialization — creato solo quando serve

L'istanza non viene creata finché qualcuno non la richiede. Se nessuno la usa, non occupa memoria.

2. Inizializzato a Runtime

Le variabili statiche classiche vengono inizializzate dal compilatore prima che il programma parta — non possono leggere configurazioni da file o dipendere da altri sistemi. Il Singleton invece viene inizializzato tardi, quando tutte le informazioni sono già disponibili.

3. Puoi fare Subclassing

Puoi avere una classe base astratta Singleton e derivarne implementazioni concrete

Perché Lo Rimpiangiamo

Il Singleton non risolve nessuno dei problemi delle variabili globali — perché è una variabile globale, solo incapsulata in una classe. I problemi sono tre:

1. Rende il codice difficile da ragionare

Una funzione che non tocca stato globale è comprensibile leggendo solo il suo corpo. Non appena chiama `SomeManager.Instance.DoSomething()`, devi tracciare tutto il codice che tocca quello stato per capire cosa sta succedendo.

2. Incoraggia il coupling

Se `AudioSystem.Instance` è accessibile ovunque, un nuovo programmatore che deve fare suonare un masso può semplicemente scrivere `AudioSystem.Instance.Play()` dentro il codice della fisica. Architettura compromessa con una riga. Se l'istanza non fosse globale, sarebbe costretto a trovare un approccio corretto.

3. Non è thread-safe

Uno stato globale accessibile da tutti i thread contemporaneamente porta a deadlock e race condition.

Risolve Due Problemi Anche Quando Ne Hai Uno Solo

Il GoF descrive il Singleton come soluzione a due problemi: istanza unica e accesso globale. Spesso ne hai bisogno solo di uno, ma il pattern te li impone entrambi.

Esempio: hai un sistema di logging. Vuoi accesso conveniente — non vuoi passare l'istanza ovunque. Usi il Singleton. Poi a metà sviluppo vuoi più logger separati per audio, UI, gameplay. Non puoi — il design dell'istanza unica è ora incrostato in ogni riga di codice:

Cosa Fare Invece

Se hai un Manager inutile → sposta la logica direttamente nella classe che gestisce.

Se hai bisogno di accesso comodo → passa l'istanza come parametro, o raggruppalala in un unico `GameManager` invece di creare Singleton separati per ogni sistema.

Se hai bisogno di istanza unica → usa un assert nel costruttore, non un Singleton completo.

Nella pratica usa classi statiche o un flag statico per garantire istanza unica. I pattern **Subclass Sandbox** e **Service Locator** coprono i casi rimanenti in modo più pulito.

Il problema centrale è uno solo:

Il Singleton è una variabile globale mascherata da pattern. Tutti i problemi delle variabili globali rimangono.

State

Cos'è il State Pattern?

Consente a un oggetto di modificare il proprio comportamento quando il suo stato interno cambia. L'oggetto sembrerà cambiare classe.

Invece di uno `switch` gigante, **delegiamo il comportamento a un oggetto separato** che rappresenta lo stato corrente. Cambiando quell'oggetto, cambia il comportamento.

La Struttura in C# per Unity

Innanzitutto, definiamo un'interfaccia per lo stato. Ogni comportamento dipendente dallo stato diventa un metodo virtuale in tale interfaccia.

I Tre Elementi dello State Pattern

1. L'Interfaccia dello Stato In primo luogo, definiamo un'interfaccia (o una classe base astratta) per lo stato. Ogni comportamento che varia a seconda dello stato in cui ci si trova diventa un metodo (spesso definito come virtuale) all'interno di questa interfaccia.

2. Una Classe per Ogni Stato Per ciascuno stato possibile, creiamo una classe dedicata che implementa l'interfaccia. I metodi al suo interno definiranno il comportamento specifico dell'entità (ad esempio, la nostra eroina) esclusivamente mentre si trova in quello stato.

3. Delega al Posto dello Switch Infine, assegniamo all'entità un riferimento al suo stato corrente. In questo modo eliminiamo i giganteschi e fragili blocchi `switch` (o le lunghe catene di `if/else`) e deleghiamo l'esecuzione del comportamento direttamente all'oggetto stato attivo in quel momento.

Riepilogo Strutturale

Elemento	Responsabilità
<code>IState</code>	Contratto comune a tutti gli stati
<code>Entity</code>	Possiede lo stato, delega comportamento
<code>StandingState</code> , <code>JumpingState</code> , ecc.	Implementano comportamento specifico
<code>ChangeState()</code>	Unico punto di transizione

Il Problema Iniziale

Le Finite State Machine (FSM) nascono per risolvere un problema comune: gestire un'entità che può trovarsi in stati diversi e mutualmente esclusivi. L'approccio naive con variabili booleane multiple (isJumping, isDucking, isDiving) fallisce perché crea combinazioni potenzialmente invalide e porta a codice fragile, pieno di condizioni annidate che si rompono ad ogni modifica.

La Prima Soluzione: Enum + Switch

Il primo miglioramento consiste nel sostituire i booleani con un enum. Una variabile di tipo enum può contenere un solo valore alla volta, eliminando by design le combinazioni invalide. Il comportamento viene gestito con uno switch sullo stato corrente.

Questo approccio funziona per casi semplici, ma scala male: se uno stato ha dati specifici (come un timer di caricamento), questi finiscono nella classe principale anche se hanno senso solo in quello stato. Aggiungere funzionalità richiede modifiche in più punti.

Il Pattern State della Gang of Four

La definizione originale recita: "Allow an object to alter its behavior when its internal state changes. The object will appear to change its class."

Il pattern si compone di tre elementi:

Un'interfaccia che definisce il contratto comune a tutti gli stati, con metodi come HandleInput e Update.

Una classe concreta per ogni stato che implementa l'interfaccia. Ogni stato incapsula il proprio comportamento e i propri dati. Un timer di caricamento vive dentro lo stato che lo usa, non nella classe principale.

Il contesto (l'entità) che possiede un riferimento allo stato corrente e delega ad esso le chiamate. L'entità non contiene switch né logica condizionale sullo stato.

Entry e Exit Actions

Un problema emerge quando lo stato uscente deve configurare lo stato entrante: si crea accoppiamento e duplicazione. Se più stati possono transitare verso Standing, ognuno deve settare la grafica corretta.

La soluzione è dare ad ogni stato metodi Enter ed Exit. Enter viene chiamato quando si entra nello stato e permette allo stato di configurare se stesso. Exit viene chiamato quando si esce e permette pulizia. Ogni stato diventa completamente autosufficiente.

Dove Vivono gli Oggetti Stato

Con l'approccio enum, i valori sono primitivi. Con il pattern State, servono istanze reali.

Stati statici: se uno stato non ha dati propri (è puro comportamento), può esistere come istanza singola condivisa. Tutte le entità che si trovano in quello stato puntano alla stessa istanza. Zero allocazioni, nessuna pressione sul Garbage Collector. Questo è il pattern Flyweight.

Stati istanziati: se uno stato ha dati specifici dell'istanza, ogni entità deve avere la propria copia. Due personaggi entrambi in Ducking devono avere timer di caricamento separati.

Concurrent State Machines

Quando un'entità ha aspetti indipendenti del proprio stato (cosa sta facendo e cosa sta portando), una singola FSM porta a esplosione combinatoria: n stati per m equipaggiamenti richiede $n \times m$ stati totali.

La soluzione è usare due FSM separate che operano in parallelo. L'entità possiede due riferimenti a stati correnti e delega gli input a entrambe. Con due macchine servono solo $n + m$ stati.

Quando le due FSM devono interagire (non puoi sparare mentre salti), si usano controlli incrociati espliciti. Non elegante ma funzionale.

Hierarchical State Machines

Quando più stati condividono comportamento comune (Standing, Walking, Running possono tutti saltare e abbassarsi), si duplica codice.

La soluzione è organizzare gli stati in gerarchia usando ereditarietà. Un superstato OnGroundState gestisce il comportamento comune. I sottostati ereditano e aggiungono logica specifica. Se il sottostato non gestisce un input, lo propaga al superstato chiamando il metodo base.

L'ereditarietà è potente ma crea accoppiamento forte tra le classi. Va usata con cautela.

Pushdown Automata

Le FSM normali non hanno memoria: sanno dove sono ma non da dove vengono. Questo è un problema per stati temporanei come sparare: dopo l'animazione, dove si torna?

La soluzione è sostituire il singolo riferimento allo stato con uno stack. Due nuove operazioni diventano disponibili:

Push aggiunge uno stato in cima allo stack. Diventa lo stato corrente ma quello precedente resta sotto, congelato.

Pop rimuove lo stato in cima. Quello sotto torna automaticamente corrente.

Per transizioni definitive si usa Change (sostituisce). Per stati temporanei si usa Push, e quando terminano fanno Pop tornando automaticamente allo stato precedente.

Quando Usare le FSM

Le FSM sono utili quando l'entità ha comportamento che cambia in base a stato interno, quello stato può essere diviso rigidamente in un numero piccolo di opzioni distinte, e l'entità risponde a input o eventi nel tempo.

Nei giochi si usano per controllo del personaggio, navigazione menu, protocolli di rete, flusso di gioco, NPC semplici.

Le FSM hanno limiti: non possono rappresentare comportamenti che richiedono memoria arbitraria, decisioni su dati complessi, o pianificazione a lungo termine. Per AI complesse servono Behavior Trees o sistemi di planning.

Tabella Riepilogativa

Variante	Problema Risolto
Enum + Switch	Stati invalidi da booleani multipli
State Pattern	Incapsulamento di comportamento e dati per stato
Entry/Exit Actions	Setup e cleanup centralizzati, niente duplicazione
Stati Statici vs Istanziati	Gestione memoria e allocazioni
Concurrent FSM	Esplosione combinatoria di aspetti indipendenti
Hierarchical FSM	Duplicazione codice tra stati con comportamento comune
Pushdown Automaton	Tornare allo stato precedente dopo azioni temporanee

DESIGN PATTERN : ESEMPI CONCRETI

Command

Uso nel Game Dev	Esempio Concreto
Input rebinding	Tasti personalizzabili dal giocatore
Undo/Redo	Editor di livelli, giochi strategici a turni
Replay system	Registra comandi, riesegui per replay
AI decision queue	NPC accoda azioni invece di eseguirle subito
Network multiplayer	Serializza comandi, invia al server
Tutorial system	Esegui comandi "finti" per mostrare al giocatore

Flyweight

Uso nel Game Dev	Esempio Concreto
Tile-based maps	Ogni cella punta a un TileTypeSO condiviso
Foreste/Vegetazione	Migliaia di alberi, una sola mesh/texture
Bullet hell	Centinaia di proiettili, stesso sprite
Item database	ItemSO condiviso tra tutti i drop uguali
Particle systems	Particelle condividono materiale
Enemy stats	EnemyDataSO base, istanze solo per posizione/HP

Observer

Uso nel Game Dev	Esempio Concreto
Achievement system	Ascolta eventi da sistemi diversi
UI updates	Healthbar ascolta evento OnHealthChanged
Audio triggers	AudioManager ascolta OnEnemyDeath
Analytics	Logga eventi senza toccare gameplay code
Quest system	Ascolta OnItemCollected, OnEnemyKilled
Save system	Ascolta OnCheckpointReached

Prototype

Uso nel Game Dev	Esempio Concreto
Prefab spawning	<code>Instantiate(prefab)</code> è Prototype nativo
Runtime cloning	Clona nemico con stato attuale (HP, buff)
Procedural generation	Clona template, modifica proprietà
Data inheritance	JSON/ScriptableObject con campo "prototype"
Editor tools	Duplica oggetti mantenendo configurazione

Singleton (e Alternative)

Problema	Soluzione Migliore
"Mi serve un GameManager globale"	Passa riferimenti espliciti, o un singolo ServiceLocator
"AudioManager deve essere accessibile ovunque"	Iniecta dipendenza, o eventi (Observer)
"Solo un'istanza di X"	Assert nel costruttore, non Singleton completo
"Lazy initialization"	Factory method o ScriptableObject
"Accesso comodo ai sistemi"	Un solo GameContext che raggruppa i riferimenti

State

Uso nel Game Dev	Esempio Concreto
Player controller	Standing, Jumping, Ducking, Attacking
Enemy AI	Patrol, Chase, Attack, Flee
Menu navigation	MainMenu, Options, Credits (Pushdown)
Game flow	Loading, Playing, Paused, GameOver
Animation logic	Completa Animator con logica custom
Network connection	Connecting, Authenticated, InLobby, InGame

Sequencing Patterns: Il Controllo del Tempo nei Videogiochi

Il Problema Centrale

I videogiochi simulano **mondi paralleli** che sembrano vivere in tempo reale, ma girano su hardware **sequenziale** (la CPU esegue un'istruzione alla volta). Questa è una tensione fondamentale:

MONDO PERCEPITO	vs	MONDO REALE
-----	-----	
Tutto accade simultaneamente		Le istruzioni vengono eseguite una per volta
Player, nemici, proiettili si muovono "insieme"		CPU: aggiorna player poi aggiorna nemico poi aggiorna proiettile poi renderizza tutto

I Sequencing Patterns Risolvono Questo

I pattern di questa sezione rispondono a domande come:

1. **Come creo l'illusione che il tempo scorra?** → Game Loop
2. **Come sincronizzo gli aggiornamenti degli oggetti?** → Update Method
3. **Come nascondo il fatto che aggiorni tutto in sequenza?** → Double Buffer

La Metafora dell'Orologio

Nystrom usa una metafora potente: **costruire un orologio**.

Un orologio meccanico ha:

- Un **meccanismo centrale** che scandisce il tempo (pendolo)
- Degli **ingranaggi** che traducono quel battito in movimento sincronizzato
- Una **facciata** che mostra l'ora come se tutto si muovesse insieme

Nei videogiochi:

- **Game Loop** = il pendolo che batte (tick costante)
- **Update Method** = gli ingranaggi che trasmettono quel tick agli oggetti
- **Double Buffer** = la facciata che nasconde il lavoro sequenziale

Double Buffer: Nascondere il Lavoro Sequenziale

Buffer = Memoria temporanea per **accumulare dati** prima di elaborarli

Il Problema Fondamentale

Il mondo del gioco deve **apparire simultaneo**, ma il computer lavora **un'istruzione alla volta**.

Il Double Buffer risolve questo problema specifico:

Come faccio a modificare uno stato incrementalmente (pixel per pixel, oggetto per oggetto) senza che nessuno veda il lavoro a metà?

L'Analogia del Teatro (Fondamentale)

Nystrom usa un'analogia perfetta. Immagina di dover cambiare la scenografia tra due scene di uno spettacolo:

Soluzione Naive (Fallisce)

SCENA 1 finisce

↓

[Luci accese]

- Scenografi entrano
- Rimuovono tavolo
- Aggiungono albero
- Aggiustano sfondo

[Pubblico vede tutto questo caos]

↓

SCENA 2 inizia

Problema: l'illusione è rotta. Il pubblico vede il "dietro le quinte".

Soluzione Double Buffer

DUE PALCHI AFFIANCATI:

PALCO A (luci accese)	PALCO B (luci spente)
-----------------------	-----------------------

Scena 1 in corso	Scenografi lavorano per preparare Scena 2
------------------	--

[Scena 1 finisce]

↓ SWAP (interruttore luci) ↓

PALCO A (luci spente)	PALCO B (luci accese)
-----------------------	-----------------------

Scenografi preparano Scena 3	Scena 2 inizia SUBITO (pubblico guarda qui)
---------------------------------	--

Risultato: il pubblico **non vede mai** il lavoro in corso. Ogni cambio scena è **istantaneo**.

Come Funziona nei Computer Graphics

Immagina di disegnare una faccia sorridente pixel per pixel:

```
void DrawFace()
{
    framebuffer[0,0] = BLACK; // Occhio sinistro
    framebuffer[3,0] = BLACK; // Occhio destro
    framebuffer[1,2] = BLACK; // Bocca inizio
    // ... il video driver LEGGE PROPRIO QUI ...
    framebuffer[2,2] = BLACK; // Bocca mezzo
    framebuffer[3,2] = BLACK; // Bocca fine
}
```

Cosa succede?

FRAME 1: Video driver legge framebuffer

Vede: occhi + metà bocca

FRAME 2: Video driver legge framebuffer

Vede: occhi + bocca intera + occhi vecchi

RISULTATO: flickering, tearing, immagine "strappata"

Il monitor scansiona costantemente la memoria del framebuffer, anche mentre tu stai scrivendo. È come se il pubblico guardasse il palco mentre i scenografi lavorano.

La Soluzione: Due Buffer

BUFFER A (current)	BUFFER B (next)
-----	-----
Video driver	Codice di rendering
LEGGE da qui	SCRIVE qui
(visualizzato sullo schermo)	(lavoro in corso, invisibile)

Regola d'oro:

- **Lettura** → sempre dal buffer **current**
- **Scrittura** → sempre sul buffer **next**
- **Swap** → scambia istantaneamente i ruoli

Perché lo Swap Deve Essere Atomico?

Atomico = tutto o niente, nessuno stato intermedio visibile.

```
private void Swap()
{
    Framebuffer temp = current; // [1] Atomico: temp = reference
    current = next;             // [2] Atomico: current = reference
    next = temp;                // [3] Atomico: next = reference
}
```

Il problema dell'Atomicità (Tutto o Niente)

Nello sviluppo, se lo scambio (swap) tra il buffer vecchio e quello nuovo non è **atomico**, c'è il rischio che un altro sistema (come il thread della scheda video) legga i dati esattamente a metà dell'operazione. Risultato? Il video driver disegna a schermo un "frankenstein", metà frame vecchio e metà frame nuovo (sfarfallio o *tearing*). In C#, cambiare l'assegnazione di un singolo riferimento è istantaneo, ma scambiare due variabili richiede tre passaggi. Se un thread si intromette tra quei tre passaggi, il sistema legge dati corrotti.

Copia dei Dati vs Swap di Puntatori (Il Trade-off Architetturale)

Questa è la decisione ingegneristica più importante. Dipende dal "peso" dei tuoi dati.

- **Swap di Puntatori (Solo per dati enormi):**
 - *Come funziona:* Non sposti i dati, scambi solo le "etichette" (i reference).
 - *Numeri reali:* Scambiare puntatori impiega circa **10 nanosecondi**. Copiare un intero Framebuffer grafico a 1080p (circa **8 MB** di dati) richiederebbe **5-10 millisecondi**. A 60 FPS hai solo **16.6 ms** in totale per fare tutto: la copia fisica brucerebbe metà del tempo disponibile per un singolo frame.
 - *Contro:* I dati del buffer "futuro" saranno vecchi di *due* frame logici, non uno.
- **Copia dei Dati (Per stati piccoli):**
 - *Come funziona:* Ricopi fisicamente il valore (es. un booleano **slapped**) dal futuro al presente.
 - *Numeri reali:* Ricopiare un singolo byte richiede frazioni di nanosecondo.
 - *Pro:* Lo stato è sempre perfettamente sincronizzato al frame precedente.

Il trucco Ibrido (L'indice condiviso): Se hai 100 comici, invece di fare 100 copie o scambiare 100 puntatori, crei un array di due slot per ogni attore `[0, 1]` e usi una variabile **static** globale per indicare quale slot è la "lettura" e quale la "scrittura". Quando fai lo swap, cambi solo l'indice globale (da 0 a 1). Ottieni la velocità istantanea dei puntatori con la comodità della memoria isolata.

L'Anti-pattern dei Lock

Usare la keyword **lock** per proteggere lo stato dei tuoi attori e prevenire le *race condition* è un errore di design devastante per tre motivi:

1. **Prestazioni distrutte:** Su 1.000 attori potresti generare migliaia di lock al secondo. Un lock "contestato" (dove un thread deve aspettare l'altro) costa cicli preziosi di CPU, causando cali di framerate.
2. **Deadlock (Il blocco totale):** Se Harry aspetta che si liberi il lock di Baldy, e Baldy sta aspettando che si liberi il lock di Harry, il gioco si congela all'infinito.
3. **Non risolve l'errore logico:** Il lock mette i thread in fila indiana per non farli schiantare, ma *non impedisce* a Baldy di reagire allo schiaffo di Harry all'interno dello stesso identico frame, che era il nostro problema di game design originale. Il Double Buffer (senza lock) risolve la radice del problema.

Due buffer: uno **visibile** (current), uno **nascosto** (next).

CURRENT (visibile)	NEXT (nascosto)
-----	-----
L'utente vede questo	Tu lavori qui (disegni la faccia)

[SWAP istantaneo quando finisci]

CURRENT (visibile)	NEXT (nascosto)
-----	-----
Ora l'utente vede la faccia completa	Prepari il prossimo frame qui

Quando Usarlo

Servono **TUTTE E 4** queste condizioni:

1. **Modifichi** qualcosa **un pezzo alla volta**
2. Qualcuno può **leggere** mentre tu **scrivi**
3. Non vuoi che vedano il **lavoro a metà**
4. Lettura e scrittura non devono **bloccarsi a vicenda**

Due Modi per Fare lo Swap

A) Scambia puntatori (veloce, per buffer grandi):

```
Framebuffer temp = current;  
current = next; // Swap istantaneo  
next = temp;
```

Pro: $O(1)$, velocissimo

Contro: dati nel buffer sono di 2 frame fa

B) Copia dati (per buffer piccoli):

```
currentValue = nextValue; // Copia il valore  
  
nextValue = 0; // Reset
```

Pro: dati sempre del frame precedente

Contro: lento se buffer è grande

Applicazioni Reali

- **Grafica:** tutti i giochi usano doppio framebuffer
- **Fisica:** entità che interagiscono (non vedono modifiche parziali)
- **AI:** comportamenti simultanei (ordine di update irrilevante)
- **Audio:** crossfade tra brani
- **Input:** evitare race conditions

Regola d'oro:

- **Leggi** sempre da `current`
- **Scrivi** sempre su `next`
- **Swap** quando finito → diventa tutto visibile insieme

Game Loop

Cos'è il Game Loop? È il cuore pulsante di ogni videogioco. A differenza dei normali software (come Word, che aspetta fermo che tu prema un tasto), il Game Loop gira continuamente all'infinito per muovere i nemici, calcolare la fisica e disegnare la grafica, anche se il giocatore non tocca il controller.

La sua struttura base è composta da tre fasi cicliche:

1. **Process Input:** Legge i comandi del giocatore (senza mai bloccarsi in attesa).
2. **Update:** Fa avanzare lo stato del gioco (muove i personaggi, calcola la fisica e l'IA).
3. **Render:** Disegna a schermo il risultato.

Il Problema del Tempo I giocatori hanno hardware diversi. Se un loop gira "il più veloce possibile", su un PC super-potente il gioco andrà a velocità folle, mentre su un PC vecchio andrà a rallentatore. Il compito cruciale del Game Loop è **scollegare la velocità del gioco dalla potenza del processore**.

Le 4 Soluzioni Architettureali:

1. **Loop Base "As fast as you can" (Sbagliato):** Il gioco gira alla massima velocità concessa dalla CPU. Ingiocabile, perché la velocità del gameplay dipende dall'hardware.
2. **Fisso con riposo / Sleep (Limitato ma utile):** Mette in pausa la CPU alla fine di ogni frame per mantenere un tetto massimo (es. 60 FPS). Ottimo per i giochi Mobile perché risparmia batteria, ma se il dispositivo è troppo lento per calcolare un frame, il gioco andrà inevitabilmente in *slow-motion*.
3. **Variable Time Step (Pericoloso):** Calcola il tempo reale passato dal frame precedente (**deltaTime**) e lo moltiplica per la velocità degli oggetti.
 - *Pro:* Fluidità perfetta su ogni schermo.
 - *Contro:* Rende il gioco **non-deterministico**. La fisica diventa instabile (gli oggetti volano via per errori di arrotondamento dei float) e in multiplayer i giocatori vedranno cose diverse sui loro schermi. Fortemente sconsigliato per la fisica.
4. **Fixed Update + Variable Render (La Soluzione Definitiva):**

È lo standard dell'industria moderna. Separa la mente (logica) dagli occhi (grafica): * **Update (Fisica/Logica):** Viene eseguito a step temporali rigorosamente **fissi** (es. esattamente ogni 16 millisecondi). Questo garantisce una fisica stabile e uguale su ogni PC. Se il PC è lento, il gioco forza cicli multipli di Update per "rimettersi in pari" col tempo reale. * **Render (Grafica):** È libero. Se la CPU fatica, il motore grafico *salta* il disegno di qualche frame (abbassando gli FPS) pur di far girare la logica del gioco alla velocità corretta. Il gameplay non va mai a rallentatore, al massimo scatta visivamente.

I Tre Pilastri del Loop di Unity

1. `FixedUpdate` (Il Dominio della Fisica)

- **Frequenza:** Costante e prevedibile (di default 50 volte al secondo, ogni 0.02s).
- **Comportamento:** È scollegato dal framerate visivo. Se il gioco subisce un calo di FPS, Unity chiamerà il `FixedUpdate` più volte nello stesso frame grafico per "mettersi in pari" con il tempo reale. Se il PC è velocissimo, potrebbe non chiamarlo affatto in un frame visivo.
- **Responsabilità Esclusive:** * Aggiungere forze a un `Rigidbody`.
 - Modificare la velocità fisica.
 - Spostare oggetti con Collider solidi.
- **Regola d'Oro:** Non usare MAI `Time.deltaTime` qui dentro. La matematica temporale è già gestita dal motore fisico.

2. `Update` (Il Dominio della Logica)

- **Frequenza:** Variabile e instabile. Viene chiamato esattamente **una volta per ogni frame disegnato a schermo**.
- **Comportamento:** Dipende totalmente dalla potenza del dispositivo. A 60 FPS gira 60 volte al secondo; su un vecchio telefono a 30 FPS, gira la metà delle volte.
- **Responsabilità Esclusive:**
 - Lettura dell'Input del giocatore (pressione di tasti e click).
 - Timer e cooldown delle abilità.
 - Logica di gioco, Intelligenza Artificiale (AI) e trigger di animazioni.
 - Movimenti puramente cinematici (senza `Rigidbody`).

3. `LateUpdate` (Il Dominio della Rifinitura)

- **Frequenza:** Variabile, esattamente come l'`Update`.
- **Comportamento:** Unity garantisce al 100% che il `LateUpdate` verrà chiamato **solo dopo** che tutti gli script di tutto il gioco hanno terminato il loro rispettivo `Update`.
- **Responsabilità Esclusive:**
 - Inseguimento della Telecamera (la telecamera deve muoversi *dopo* che il giocatore ha calcolato il suo movimento, altrimenti l'immagine tremerà).
 - Aggiustamenti procedurali finali (es. far girare la testa del personaggio verso un nemico dopo che l'animazione di corsa è finita).

Il Compensatore Universale: `Time.deltaTime`

È la variabile più importante per la logica di gioco. Rappresenta il **tempo in secondi trascorso dall'ultimo frame visivo**.

- **Il Problema:** Se muovi un oggetto di 5 metri in `Update`, a 60 FPS si muoverà di 300 metri in un secondo. A 30 FPS si muoverà solo di 150 metri. Il gioco cambia a seconda dell'hardware.
- **La Soluzione:** Moltiplicando la tua velocità per `Time.deltaTime`, trasformi l'istruzione da *"muoviti di 5 metri ogni frame"* a *"muoviti di 5 metri al secondo"*. Il movimento diventa fluido e identico su qualsiasi schermo.

Update Method

Il Core del Pattern: "Update Method"

Il Problema (Il Loop Monolitico): In un approccio *naive*, inserire il comportamento sequenziale degli oggetti (es. un ciclo `for` per muovere un nemico) direttamente nel Game Loop centrale blocca l'intero programma. Lo schermo si congela finché l'azione non è finita. Inoltre, mischiare la logica di cento entità diverse nel loop principale crea un'architettura rigida ("mushed") impossibile da scalare.

In ingegneria del software, l'**approccio naive** (tradotto letteralmente come *ingenuo* o *semplicitico*) è la **prima soluzione, la più ovvia e immediata**, che ti viene in mente per risolvere un problema, senza preoccuparti delle conseguenze a lungo termine.

È l'implementazione "forza bruta". Di solito, la soluzione naive ha queste caratteristiche:

- **Ignora il contesto sistemico:** Risolve il problema locale ma non tiene conto dell'architettura generale, ignorando concetti come la scalabilità, la memoria o i colli di bottiglia della CPU.
- **Funziona "nel vuoto":** Se la testi in isolamento, funziona perfettamente. Ma non appena la inserisci in un videogioco complesso con migliaia di entità, fa crollare il framerate, causa bug imprevedibili o rende il codice illeggibile.

La Soluzione (Delega del Comportamento): Il Game Loop viene svuotato da ogni logica specifica. Si limita a mantenere una collezione di entità e a chiamare un singolo metodo – l'`Update()` – su ciascuna di esse una volta per frame. Ogni entità incapsula il *proprio* comportamento e avanza la sua simulazione di un solo passo logico alla volta.

Il Contesto in Unity

Unity è costruito interamente attorno a questo pattern. L'engine nasconde il Game Loop primario (scritto in C++) e ti espone l'interfaccia dell'Update Method tramite il ciclo di vita dei `MonoBehaviour` (il metodo `Update()`).

Vantaggi Architettureali

- **Separazione delle Responsabilità:** Il loop coordina il tempo, l'entità gestisce se stessa.
- **Scalabilità Estrema ($O(1)$ per l'aggiunta):** Aggiungere un nuovo nemico o rimuoverne uno richiede solo di modificare la lista centrale, senza toccare il motore del gioco.
- **Data-Driven:** Permette di istanziare oggetti dinamicamente (es. da un file JSON o dall'Inspector) senza ricompilare il Game Loop.

Summary

I 3 Pattern Sequenziali (Cheat Sheet Architetture)

1. Game Loop (Il Motore del Tempo)

- **Il Problema:** Scollegare la fluidità del gioco dalla potenza dell'hardware del PC.
- **La Soluzione (Standard dell'Industria):** *Fixed Update + Variable Render*. La fisica e la logica core si aggiornano a intervalli di tempo rigidamente fissi (es. 16ms). Il rendering grafico disegna i frame a schermo non appena la GPU è pronta, indipendentemente dalla logica.

2. Update Method (Delega dei Comportamenti)

- **Il Problema:** Un loop monolitico centrale non scala e crea codice "spaghetti".
- **La Soluzione:** Il loop centrale si limita a iterare su una lista di entità, chiamando il metodo `Update()` su ciascuna. Ogni oggetto incapsula e gestisce il proprio comportamento frame per frame.

3. Double Buffer (Sincronizzazione di Memoria)

- **Il Problema:** Evitare glitch o *race conditions* leggendo dati mentre un altro sistema li sta ancora calcolando/scrivendo.
- **La Soluzione:** Mantenere due stati paralleli. Si legge solo dal buffer `current`, si scrive solo nel buffer `next`. A fine frame, si esegue lo `swap`.
 - *Swap Puntatori:* Per dati pesanti (es. framebuffer grafici). Istantaneo.
 - *Copia Dati:* Per dati leggeri (es. bool, struct). Semplice e sicuro.

La Trasposizione in Unity (Il Tuo Toolkit)

Unity nasconde il Game Loop sottostante (scritto in C++) e ti espone il pattern *Update Method* tramite il ciclo di vita dei `MonoBehaviour`:

- **FixedUpdate:** Frequenza fissa (es. 0.02s). Esclusivo per **Fisica, Forze e Rigidbody**. Regola: *NON usare mai `Time.deltaTime` qui.*
- **Update:** Frequenza variabile. Esclusivo per **Logica, Intelligenza Artificiale, Timer e Input**. Regola: *Moltiplicare sempre le trasformazioni spaziali per `Time.deltaTime` per scollegarle dal framerate.*
- **LateUpdate:** Frequenza variabile, ma eseguito matematicamente per ultimo. Esclusivo per **Telecamera e aggiustamenti procedurali** (es. ossa e animazioni).

Conclusione: Unity gestisce tutto, tu usi solo `Update()` e `Time.deltaTime`.

Behavioral Patterns: Overview e Roadmap

Il problema centrale

I giochi richiedono una **quantità enorme di comportamenti diversi** rispetto ad altri tipi di software.

Esempio concreto: Un word processor ha molte feature, ma un RPG medio ha:

- Decine/centinaia di tipi di nemici
- Centinaia di oggetti con comportamenti unici
- Migliaia di quest con logiche diverse

La sfida: Come definire questa **massa di comportamenti** senza:

- Scrivere migliaia di classi
- Perdere manutenibilità
- Rendere impossibile l'iterazione rapida

I tre pattern proposti

Perché questi pattern sono "Behavioral"?

Tutti e tre affrontano lo stesso problema fondamentale:

Come gestire la scala e la varietà dei comportamenti in un gioco?

- **Type Objects:** Scala tramite classificazione basata su dati
- **Subclass Sandbox:** Scala tramite composizione di primitive
- **Bytecode:** Scala tramite separazione codice/dati

"Categorie di comportamento senza rigidità di classi"

Invece di:

```
class Goblin : Enemy { }
class Orc : Enemy { }
class Dragon : Enemy { }
// ... 200 classi nemici
```

Fai:

```
class Enemy {
    EnemyType type; // Type Object
}
```

Beneficio: Crei nuovi tipi di nemici modificando **dati**, non codice.

2. Subclass Sandbox

"Set sicuro di primitive per definire comportamenti"

Invece di lasciare che ogni behavior faccia qualsiasi cosa, fornisci un **toolbox limitato** di operazioni.

Analogia: Dare ai designer LEGO invece di materie prime grezze.

Beneficio: I comportamenti complessi si compongono da operazioni semplici e controllate.

3. Bytecode

"Comportamento come dati, non come codice"

Sposta la logica di comportamento **fuori dal codice compilato** in:

- Script interpretati
- Linguaggi DSL (Domain-Specific Language)
- Dati strutturati eseguibili

Beneficio: Designer/scripter modificano comportamenti senza ricompilare.

Perché questi pattern sono "Behavioral"?

Tutti e tre affrontano lo stesso problema fondamentale:

Come gestire la scala e la varietà dei comportamenti in un gioco?

- **Type Objects:** Scala tramite classificazione basata su dati
- **Subclass Sandbox:** Scala tramite composizione di primitive
- **Bytecode:** Scala tramite separazione codice/dati

Bytecode

Bytecode = Istruzioni eseguibili codificate come dati

Il Problema Fondamentale

Immagina di avere un gioco con **centinaia di comportamenti** da definire (spell, AI, dialoghi, quest logic).

Opzione A: Codice C++/C#

```
// Spell 1

public class Fireball : Spell {

    public override void Execute() {

        DealDamage(target, 50);

        PlaySound("explosion");

    }

}

// Spell 2

public class Heal : Spell { /* ... */ }

// ... altre 498 spell
```

Problemi:

- Ogni spell richiede una nuova classe C#
- Ogni modifica richiede **ricompilazione**
- Designer dipendono da programmatori
- **Impossibile moddare** (non puoi dare codice compilato agli utenti)
- **Rischio sicurezza** (codice user-generated può crashare il gioco)

Opzione B: Dati Interpretati (Interpreter Pattern)

```
// Spell come albero di oggetti

AddExpression add = new AddExpression(

    new GetHealth(wizard),

    new LiteralNumber(10)

);
```

Problemi:

- Centinaia di piccoli oggetti allocati
- Pointer jumping = **cache miss continui**
- Virtual method calls = **performance terribile**
- **68 bytes** per una semplice espressione $(1 + 2) * (3 - 4)$

La Soluzione: Bytecode

Ibrido tra codice nativo e dati interpretati.

Definisci comportamenti come **sequenza di byte** eseguita da una **Virtual Machine (VM)** che tu controlli.

Vantaggi:

- **Dense:** Blob binario compatto, zero spreco
- **Linear:** Istruzioni consecutive, cache-friendly
- **Low-level:** Ogni istruzione fa una cosa minima
- **Safe:** La VM controlla cosa il bytecode può fare
- **Fast:** Molto più veloce dell'Interpreter pattern

Analogia: La Macchina del Caffè

Immagina una **macchina del caffè automatica**. Tu non programmi in C# ogni volta che vuoi un caffè. Invece:

1. **Inserisci una capsula** (dati)
2. **Premi un pulsante** (esegui)
3. La macchina **interpreta la capsula** e fa il caffè

Bytecode è la stessa cosa: invece di scrivere codice C#, crei "capsule" (file di dati) che la VM "esegue".

Il Bytecode decompone comportamenti complessi in **istruzioni atomiche**.

Ogni istruzione fa una cosa sola, facile da comprendere e debuggare.

In Breve

Il pattern **Bytecode** trasforma la logica del gioco in un array di istruzioni codificate, lette ed eseguite a runtime da una **Virtual Machine** (VM) interna tramite uno **Stack** di memoria.

- **Il Vantaggio Principale:** Permette a designer e modder di modificare e testare istantaneamente centinaia di comportamenti complessi senza dover ricompilare il codice C#. La VM funge da "recinto di sicurezza" (Sandboxing), impedendo a chi crea i contenuti di inserire loop infiniti o far crashare il gioco.
- **Il Trade-off:** A fronte di una flessibilità totale e un'iterazione rapidissima, il Bytecode risulta più lento del codice nativo ed è complesso da implementare (richiede di programmare sia la VM che gli authoring tool). È perfetto per sistemi di abilità, dialoghi o modding, ma va evitato per logiche semplici o calcoli dove le performance sono critiche.

Subclass Sandbox

Centralizzare il coupling nella base class in modo che le sottoclassi ne siano isolate.

Ci sono due tipi di metodi nella base class:

1. La **sandbox method** — **abstract**, che le sottoclassi *devono* implementare. E' il punto dove ogni sottoclasse definisce il proprio comportamento specifico.
2. Le **provided operations** — **protected** non virtuali, che le sottoclassi *possono* usare. Sono gli strumenti messi a disposizione dalla base class.

Una **base class** definisce un **metodo astratto (la sandbox method)** che le sottoclassi devono implementare, e fornisce un insieme di **operazioni protected non-virtuali** (le provided operations) che le sottoclassi usano per implementarla.

Le sottoclassi non parlano mai direttamente con i sistemi esterni. Parlano solo con la base class.

I due ingredienti:

1. Sandbox method — **abstract, protected**. E' il "dove metti il tuo comportamento". Ogni sottoclasse la implementa a modo suo.

2. Provided operations — **protected, non-virtual**, con implementazione concreta nella base class. Sono gli strumenti. La base class le implementa e le mette a disposizione delle sottoclassi.

Il **coupling** con i sistemi esterni (AudioManager, PhysicsEngine, ParticleSystem) e' concentrato **in un solo posto**: la base class.

Le sottoclassi sono isolate. Dipendono solo dalla base class.

Problema che risolve

Hai molte sottoclassi che devono tutte interagire con gli stessi sistemi esterni (audio, fisica, particelle). Se ogni sottoclasse chiama direttamente quei sistemi, ottieni:

- **Coupling diffuso** — ogni sottoclasse dipende da ogni sistema esterno
- **Codice duplicato** — la stessa logica ripetuta in ogni sottoclasse
- **Fragilita'** — se un sistema esterno cambia, devi toccare tutte le sottoclassi

Provided operations direttamente sulla base class o su oggetti helper?

Soluzione

Centralizzare il coupling nella base class. Le sottoclassi non parlano mai con i sistemi esterni — parlano solo con la base class.

Struttura

Due ingredienti obbligatori:

1. Sandbox Method

- `abstract + protected`
- Le sottoclassi la implementano — e' il loro "posto di lavoro"
- Non e' chiamabile dall'esterno

2. Provided Operations

- `protected + non-virtual`
- Implementate dalla base class
- Le sottoclassi le usano, non le sovrascrivono
- Qui vive tutto il coupling con i sistemi esterni

3. Punto di ingresso pubblico

- Un metodo `public` sulla base class che chiama internamente la sandbox method
 - E' l'unica cosa che il mondo esterno conosce
-

Decisioni di design

Quante provided operations nella base class?

Situazione	Cosa fare
Operazione usata da una sola sottoclasse	Lasciarla fuori, non vale il costo
Operazione che legge stato senza modificarlo	Coupling "safe", puoi lasciarla fuori
Operazione che modifica stato	Candidato forte per provided operation

Trade-off del pattern

Vantaggio

Sottoclassi isolate dai sistemi esterni

Modifica a un sistema esterno tocca solo la base class

Codice duplicato ridotto nelle sottoclassi

Svantaggio

Base class accumula coupling con tutto

Base class fragile — difficile da cambiare

Gerarchia larga ma accoppiata alla base

Quando usarlo

- Hai molte sottoclassi con comportamento variabile
 - Le sottoclassi hanno bisogno degli stessi sistemi esterni
 - Vuoi che aggiungere una nuova sottoclasse non richieda toccare nulla di esterno
-

Type Object

Hai un gioco con centinaia di razze di mostri. La soluzione OOP classica:

Monster (base class)

```
|— Dragon
|— Troll
|— Zombie
|— ZombieDragon
|— ... (altri 96)
```

Ogni razza e' una **sottoclasse**. Ogni sottoclasse ha i suoi dati **hardcoded** nel codice.

Il problema non e' tecnico — e' **di processo**:

Azione	Cosa richiede
Aggiungere una nuova razza	Scrivere una nuova classe, ricompilare
Modificare la salute di un Troll	Aprire il codice, modificare, ricompilare
Creare 50 varianti di Troll	50 nuove classi, 50 file, 50 ricompilazioni

Il designer e' bloccato dal programmatore per ogni piccola modifica. I dati sono **sepolti nel codice**.

La Soluzione

Invece di una classe per ogni tipo, hai **due classi**:

- **Monster** — l'istanza concreta nel gioco
- **Breed** — contiene i dati condivisi tra tutti i mostri di quella razza

Monster → ha un riferimento a → Breed

Niente ereditarieta'. Due classi, punto.

Ogni **Breed** e' un **oggetto che rappresenta un tipo**. Puoi crearne quanti ne vuoi a runtime, caricarli da file JSON, da **ScriptableObject** — senza toccare il codice.

Ereditarieta' tra Breed

Il problema: hai 50 varianti di Troll. Tutte hanno la stessa salute base, ma attack diverso. Se vuoi cambiare la salute base, devi toccare 50 Breed.

La soluzione: le Breed possono avere una **Breed parent**, esattamente come le classi hanno una base class. Se un attributo non e' definito nella Breed figlia, lo eredita dal parent.

Approccio A — Ereditarieta' dinamica

Ogni volta che chiedi un attributo, risale la catena fino a trovarlo:

```
public class Breed
{
    private Breed _parent;
    private int _health;
    private string _attack;

    public Breed(Breed parent, int health, string attack) => (_parent, _health, _attack) =
    (parent, health, attack);

    public int GetHealth()
    {
        if (_health != null || _parent == null)
            return _health;
        return _parent.GetHealth();
    }

    public string GetAttack()
    {
        if (_attack != null || _parent == null)
            return _attack;

        return _parent.GetAttack();
    }
}
```

Approccio B — Copy-down

I valori ereditati vengono copiati nella Breed figlia al momento della costruzione. Nessuna catena da risalire:

```
public class Breed
{
    public int Health { get; private set; }
    public string Attack { get; private set; }

    public Breed(Breed parent, int health, string attack)
    {
        Health = (health == 0 && parent != null) ? parent.Health : health;
        // Per le stringhe, l'operatore ?? (null-coalescing) è perfetto.
        // Se anche quello è null (o parent è null), assegna null (o una stringa vuota).
        // Se 'attack' è null, prova a prendere 'parent.Attack'.
        Attack = attack ?? parent?.Attack ?? "Default Attack";
    }
}
```

Uso pratico dei due approcci:

```
// BASE BREED - nessun parent
Breed trollBase = new Breed(null, 25, "The troll hits you!");

// BREED FIGLI - eredita health da trollBase, override solo attack
Breed trollArcher = new Breed(trollBase, 0, "The troll archer fires an arrow!");

// trollArcher.GetHealth() -> 25 (ereditato)
// trollArcher.GetAttack() -> "The troll archer fires an arrow!" (override)
```

	Dinamica	Copy-down
Velocita' accesso	Lenta	Veloce
Aggiornamento runtime	Si	No
Memoria	Tiene il parent	Non tiene il parent

1. Compile Time in C#

In C# il compilatore non produce codice macchina direttamente. Produce **IL (Intermediate Language)** — un linguaggio intermedio indipendente dall'architettura.

Codice C# → Compilatore Roslyn → IL (.dll) → JIT → Codice Macchina

In Unity specificamente:

Unity usa **Mono** o **IL2CPP** a seconda della piattaforma:

Backend	Come funziona	Quando si usa
Mono	JIT — compila IL a runtime	Editor, sviluppo
IL2CPP	Converte IL in C++, poi compila	Build finale, iOS, console

Il JIT (Just In Time) compila il codice IL in codice macchina **la prima volta che viene eseguito**. Questo spiega perché in Unity il primo frame dopo il caricamento di una scena può avere uno spike di CPU.

2. Runtime e Memoria

In C# la memoria è divisa in **due aree fondamentali**:

Stack

- Memoria contigua, gestita automaticamente
- Contiene: variabili locali, parametri di metodo, value types (**int**, **float**, **struct**)
- Allocazione e deallocazione istantanea — LIFO (Last In First Out)
- Dimensione limitata (~1MB per thread)

Heap

- Memoria dinamica, gestita dal Garbage Collector
- Contiene: tutti i reference types (**class**, **string**, array)
- Allocazione più lenta — il GC deve trovare spazio libero

Garbage Collector in Unity

Il GC in Unity usa un algoritmo **mark-and-sweep**:

1. **Mark** — parte dalle radici (variabili statiche, stack) e marca tutti gli oggetti raggiungibili
2. **Sweep** — tutto ciò che non è marcato viene deallocato

Il problema in Unity: il GC è **stop-the-world** — mentre raccoglie, il gioco si ferma. Questo produce i famosi **GC spikes** nei profiler.

```

private List<Enemy> _enemies = new List<Enemy>(); // Allocate in awake

private StringBuilder _sb = new StringBuilder();

private void Update()
{
    _enemies.Clear();
    _sb.Clear();
    _sb.Append("Health: ").Append(_health);
}

```

Regola pratica in Unity: zero allocazioni nell'**Update**. Alloca in **Awake** o **Start**, riusa in **Update**.

Ciclo di vita di un oggetto C# in Unity

```

new Monster()
    -> allocazione heap
    -> costruttore eseguito
    -> oggetto in uso
    -> riferimento rimosso (_monster = null)
    -> GC marca come irraggiungibile
    -> GC dealloca (momento non determinabile)

```

Il momento della distruzione non e' determinabile — dipende da quando il GC decide di girare. Per questo in Unity usi **OnDestroy()** per cleanup esplicito, non il distruttore C#.

```

public class Monster : MonoBehaviour
{
    private void OnDestroy()
    {}
}

```

3. Load Time in Unity

Quando Unity carica una scena:

File .unity su disco

- Unity legge il file
- Deserializza GameObject e Component
- Alloca in memoria
- Chiama Awake() su tutti i component
- Chiama OnEnable()
- Primo frame → Start()
- Game Loop inizia

ScriptableObject e' diverso — non vive nella scena:

.asset su disco

- Caricato in memoria una volta sola
- Condiviso tra tutte le scene che lo referenziano
- Non viene distrutto al cambio scena
- Distrutto solo quando esplicitamente scaricato o al quit

Questo e' esattamente perche' li usi come Flyweight in DEAD AIR — una **AudioClipLibrary** caricata una volta, referenziata da tutti i Manager.

Asset Bundle e Addressables estendono questo sistema permettendo di caricare e scaricare asset a runtime in modo esplicito — utile per giochi con molto contenuto dove non puoi tenere tutto in memoria.

Tabella Finale — Memoria in Unity

Tipo	Dove vive	Chi gestisce	Durata
<code>int, float, struct</code>	Stack	Automatica	Scope del metodo
<code>class, array</code>	Heap	Garbage Collector	Fino al GC
<code>MonoBehaviour</code>	Heap + Scena	Unity + GC	Vita della scena
<code>ScriptableObject</code>	Heap + Asset	Unity	Fino a unload esplicito
Asset statici	RAM (dopo load)	Unity	Vita dell'applicazione

Il Tipo Puo' Cambiare a Runtime?

Il libro pone questa domanda: dopo che un **Monster** e' stato creato con una **Breed**, possiamo cambiare la sua **Breed** in un secondo momento?

Esempio concreto: un mostro muore e diventa uno zombie. Due approcci:

1. Distruggi il mostro, crea un nuovo **Monster** con **ZombieBreed**
2. Prendi il mostro esistente e cambia la sua **Breed** a **ZombieBreed**

Se il tipo NON cambia:

```
public class Monster
{
    // readonly - impossibile cambiare dopo la costruzione

    // comunica chiaramente l'intenzione

    private readonly Breed _breed;

    internal Monster(Breed breed)
    {
        _breed = breed;

        _currentHealth = breed.GetHealth();
    }
}
```

Vantaggi:

- Piu' semplice da ragionare — un mostro e' sempre quello che e' stato creato
- Piu' facile da debuggare — la breed non cambia mai sotto i tuoi piedi

Svantaggio:

- Devi creare un nuovo oggetto ogni volta — costo di allocazione

Se il tipo PUO' cambiare:

```
public class Monster:
{
    private Breed _breed;
    private int _currentHealth;

    internal Monster(Breed breed)
    {

        _breed = breed;
        _currentHealth = breed.GetHealth();
    }

    // Permette di cambiare tipo a runtime
    // Richiede validazione - il nuovo tipo potrebbe avere assunzione diverse
    public void ChangeBreed(Breed newBreed)
    {
        _currentHealth = Mathf.Min(_currentHealth, newBreed.GetHealth());
        _breed = newBreed;
    }
}
```

Vantaggi:

- Nessuna allocazione — riusi l'oggetto esistente
- Puoi preservare stato specifico dell'istanza (posizione, inventario, ecc.)

Svantaggio:

- Devi validare che il nuovo tipo sia compatibile con lo stato corrente
- Piu' difficile da debuggare — la breed puo' cambiare in qualsiasi momento

Regola pratica: usa **readonly** per default. Permetti il cambio solo se hai un motivo concreto e sei pronto a gestire la validazione.